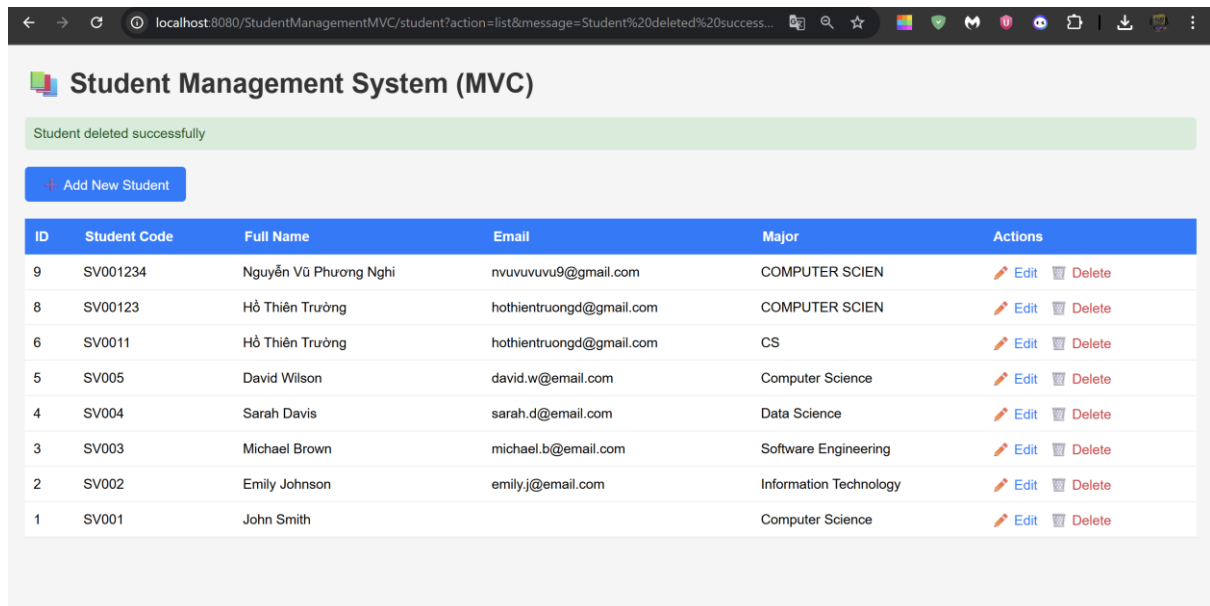


Student name: Hồ Thiên Trường – ITCSIU22306

LAB 5

Exercise 1:



- Flow explain:

- The browser initiates a **GET request** to the /student endpoint.
- This request is routed to the **StudentController** servlet mapped to /student.
- Inside the controller, the **doGet()** method handles the incoming request.
- The controller retrieves the “**action**” parameter from the URL; since it isn’t provided, it defaults to “**list**”.
- The controller then calls **listStudents(request, response)**, which in turn uses **StudentDAO.getAllStudents()** to fetch all student records from the database.

```
77 private void listStudents(HttpServletRequest request, HttpServletResponse response)
78     throws ServletException, IOException {
79
80     List<Student> students = studentDAO.getAllStudents();
81     request.setAttribute("students", students);
82
83     RequestDispatcher dispatcher = request.getRequestDispatcher("/views/student-list.jsp");
84     dispatcher.forward(request, response);
85 }
86
```

- The StudentDAO connects to the MySQL database, runs **SELECT * FROM students**, and returns a **List<Student>** object.

```

23 public List<Student> getAllStudents() {
24     List<Student> students = new ArrayList<>();
25     String sql = "SELECT * FROM students ORDER BY id DESC";
26
27     try (Connection conn = getConnection();
28         Statement stmt = conn.createStatement();
29         ResultSet rs = stmt.executeQuery(sql)) {
30
31         while (rs.next()) {
32             Student student = new Student();
33             student.setId(rs.getInt("id"));
34             student.setStudentCode(rs.getString("student_code"));
35             student.setFullName(rs.getString("full_name"));
36             student.setEmail(rs.getString("email"));
37             student.setMajor(rs.getString("major"));
38             student.setCreatedAt(rs.getTimestamp("created_at"));
39
40             students.add(student);
41         }

```

- The controller adds this list to the request object using `request.setAttribute("students", students)`.
- It then forwards the request to `/views/student-list.jsp` using a `RequestDispatcher`.

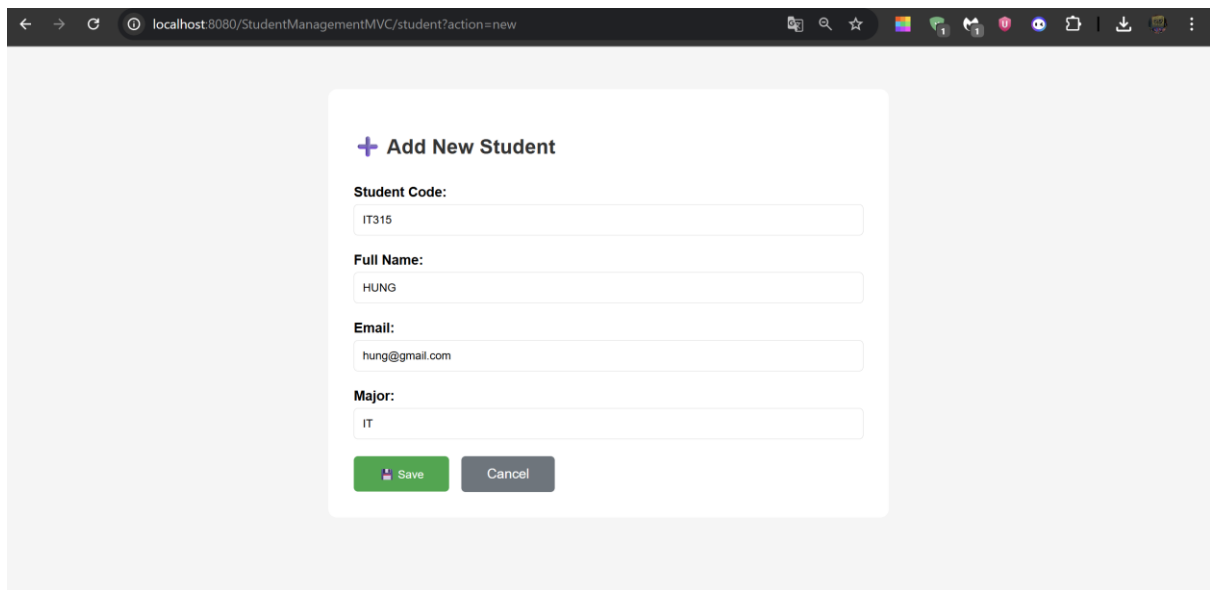
```

84 RequestDispatcher dispatcher = request.getRequestDispatcher("/views/student-list.jsp");
85 dispatcher.forward(request, response);
86
87 private void showNewForm(HttpServletRequest request, HttpServletResponse response)
88     throws ServletException, IOException {
89
90     RequestDispatcher dispatcher = request.getRequestDispatcher("/views/student-form.jsp");
91     dispatcher.forward(request, response);
92 }
93

```

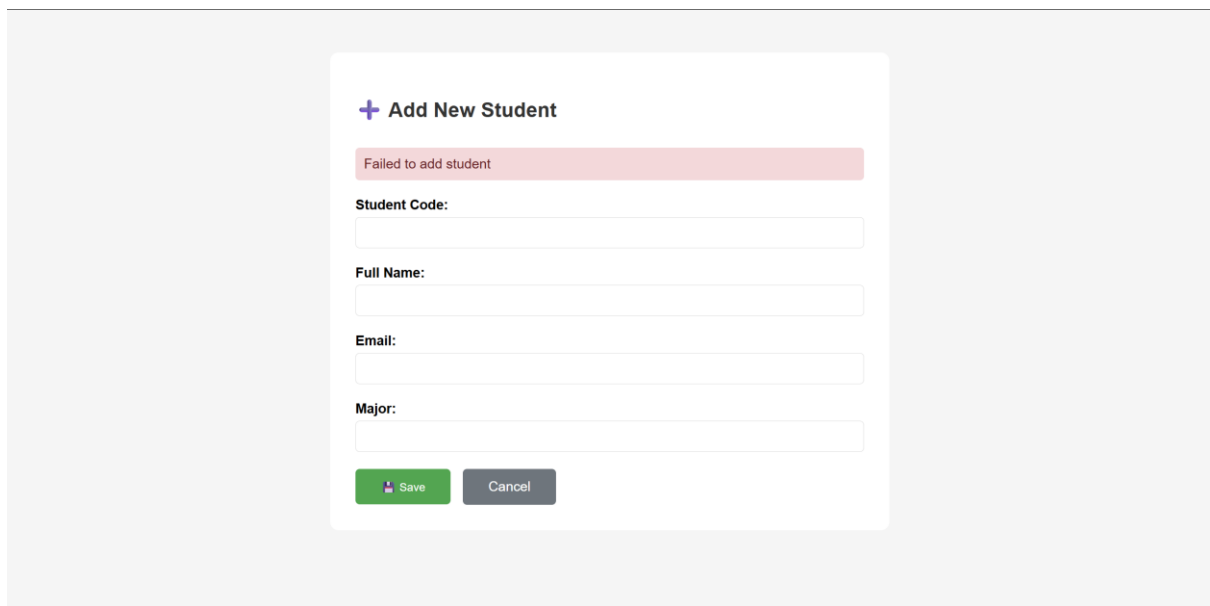
- The JSP uses JSTL (`<c:forEach>`) to loop through `${students}` - the attribute `students` which was set in the controller now can be used to loop and display each row in an HTML table,
- The rendered HTML page with the list of students is sent back as the response to the browser.

Exercise 2:



A screenshot of a web browser window at localhost:8080/StudentManagementMVC/student?action=new. The browser's address bar and various extension icons are visible at the top. The main content area displays a white form titled '+ Add New Student'. The form contains four input fields: 'Student Code:' with the value 'IT315', 'Full Name:' with the value 'HUNG', 'Email:' with the value 'hung@gmail.com', and 'Major:' with the value 'IT'. At the bottom of the form are two buttons: a green 'Save' button with a small icon and a grey 'Cancel' button.

CASE failed:



A screenshot of a web browser window showing the same 'Add New Student' form as above, but with a failure state. A red error message 'Failed to add student' is displayed in a pink box at the top of the form. The input fields for 'Student Code:', 'Full Name:', 'Email:', and 'Major:' are now empty. The 'Save' and 'Cancel' buttons remain at the bottom of the form.

Case success:

Student Management System (MVC)					
Student added successfully					
Add New Student					
ID	Student Code	Full Name	Email	Major	Actions
16	IT315	HUNG	hung@gmail.com	IT	Edit Delete
9	SV001234	Nguyễn Vũ Phương Nghi	nvuvuvu9@gmail.com	COMPUTER SCIEN	Edit Delete
8	SV00123	Hồ Thiên Trường	hothientruongd@gmail.com	COMPUTER SCIEN	Edit Delete
6	SV0011	Hồ Thiên Trường	hothientruongd@gmail.com	CS	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
1	SV001	John Smith		Computer Science	Edit Delete

Flow Explanation (Add New Student)

1. The user clicks “**Add New Student**” in the browser, which triggers a **GET** request to `/student?action=new`.
2. The **StudentController** receives this request in the **doGet()** method and recognizes the action as “**new**”.
3. The controller executes **showNewForm(request, response)**, forwarding the request to the JSP page `/views/student-form.jsp`.
4. Since no existing student data is provided, the form’s hidden field is automatically set to `<input type="hidden" name="action" value="insert">`.
5. The user fills in the input fields and clicks **Save**, which sends a **POST** request to `/student` containing all form data, including `action=insert`.
6. The controller’s **doPost()** method reads the action parameter and calls **insertStudent(request, response)**.
7. Inside **insertStudent()**, the form data (`studentCode`, `fullName`, `email`, and `major`) are extracted and used to create a new **Student** object.
8. The controller calls **studentDAO.addStudent(student)**, which connects to the MySQL database and performs an **INSERT** operation.
9. If the insertion is successful, the controller redirects to `/student?action=list&message=Student added successfully`.
10. The browser then issues a new **GET** request, the controller’s **listStudents()** runs again, and the updated list of students along with a success message is displayed on `student-list.jsp`.

Exercise 3:

Edit Student

Student Code:

Full Name:

Email:

Major:

Update successfully:

Student Management System (MVC)

Student updated successfully

Add New Student

ID	Student Code	Full Name	Email	Major	Actions
17	IT135	HUNG	hung@gmail.com	it	Edit Delete
16	IT315	HUNG	hung@gmail.com	IT	Edit Delete
9	SV001234	Nguyễn Vũ Phương Nghi	nvuvuvu9@gmail.com	COMPUTER SCIEN	Edit Delete
8	SV00123	Hồ Thiên Trường	hothientruongd@gmail.com	COMPUTER SCIEN	Edit Delete
6	SV0011	Hồ Thiên Trường	hothientruongd@gmail.com	CS	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
1	SV001	John Smith		Computer Science	Edit Delete

Flow Explanation (Edit Student)

- When the user clicks the **“Edit”** link in the browser, it triggers a **GET** request like `/student?action=edit&id=1` (from `student-list.jsp`, where the link is defined as `student?action=edit&id=${student.id}`).
- The **StudentController** receives this request in the **doGet()** method and detects that the action is **"edit"**.
- The controller then calls **showEditForm(request, response)**.
- Inside **showEditForm()**, it retrieves the student ID from the URL using `request.getParameter("id")`, and then calls **studentDAO.getStudentById(id)**.

5. The **StudentDAO** connects to the MySQL database, executes `SELECT * FROM students WHERE id = ?`, and returns the corresponding **Student** object.
6. The controller stores this object in the request scope with `request.setAttribute("student", student)`.
7. The request is then forwarded to `/views/student-form.jsp`.
8. In **student-form.jsp**, since `${student}` is not null, all form fields are prefilled with the student's existing data, and the hidden field automatically becomes `<input type="hidden" name="action" value="update">`.
9. When the user updates the information and clicks **Update**, the browser sends a **POST** request to `/student` containing the form data along with `action=update`.
10. The controller's **doPost()** method reads this action and calls **updateStudent(request, response)**.
11. The **StudentDAO** executes an **UPDATE** SQL statement to modify the student record in the database.
12. If the update is successful, the controller redirects to `/student?action=list&message=Student updated successfully`, prompting the browser to send a new **GET** request and display the refreshed student list with a success message.

Exercise 4:

Student Management System (MVC)					
Student deleted successfully					
Add New Student					
ID	Student Code	Full Name	Email	Major	Actions
16	IT315	HUNG	hung@gmail.com	IT	Edit Delete
9	SV001234	Nguyễn Vũ Phương Nghi	nvuvuvu9@gmail.com	COMPUTER SCIEN	Edit Delete
8	SV00123	Hồ Thiên Trường	hothientruongd@gmail.com	COMPUTER SCIEN	Edit Delete
6	SV0011	Hồ Thiên Trường	hothientruongd@gmail.com	CS	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
1	SV001	John Smith		Computer Science	Edit Delete

Flow Explanation (Delete Student)

1. When the user clicks the “ Delete” link in the browser, it triggers a **GET** request such as
/student?action=delete&id=1, defined in student-list.jsp as
student?action=delete&id=\${student.id}.
2. The **StudentController** receives this request in the **doGet()** method and identifies the action as **"delete"**.
3. The controller calls **deleteStudent(request, response)** to handle the deletion.
4. Inside this method, it retrieves the student's ID from the request using
int id = Integer.parseInt(request.getParameter("id"));.

```

144 private void deleteStudent(HttpServletRequest request, HttpServletResponse response)
145     throws IOException {
146
147     int id = Integer.parseInt(request.getParameter("id"));
148
149     if (studentDAO.deleteStudent(id)) {
150         response.sendRedirect("student?action=list&message=Student deleted successfully");
151     } else {
152         response.sendRedirect("student?action=list&error=Failed to delete student");
153     }
154 }
155

```

Update

```

123 private void updateStudent(HttpServletRequest request, HttpServletResponse response)
124     throws IOException {
125
126     int id = Integer.parseInt(request.getParameter("id"));
127     String fullName = request.getParameter("fullName");
128     String email = request.getParameter("email");
129     String major = request.getParameter("major");
130
131     Student student = new Student();
132     student.setId(id);
133     student.setFullName(fullName);
134     student.setEmail(email);
135     student.setMajor(major);
136
137     if (studentDAO.updateStudent(student)) {
138         response.sendRedirect("student?action=list&message=Student updated successfully");
139     }
140 }
141

```

5. The controller then calls **studentDAO.deleteStudent(id)** to remove the corresponding record from the database.
6. Within the **StudentDAO**, an SQL command
DELETE FROM students WHERE id = ?
is executed through a **PreparedStatement**.
7. The database deletes the record (if it exists) and returns a **true** result to the controller.
8. If the deletion is successful, the controller redirects to
/student?action=list&message=Student deleted successfully;
otherwise, it redirects with an error message.
9. The browser then issues a new **GET** request to /student?action=list.

10. The controller executes **listStudents()**, retrieves the updated list of students from the database, and forwards the data to `student-list.jsp`.
11. Finally, the JSP renders the updated table — the deleted student no longer appears — and a success message confirms that the deletion was completed.